

راهنمای توسعه‌دهنده: کالبدشکافی کد در سیستم‌های الهام‌گرفته از کوانتوم (۲۰۲۵-۲۰۲۶)

در این درسنامه، مستقیماً وارد لایه‌ی پیاده‌سازی می‌شویم. ما از دو ابزار قدرتمند `quimb` و `PennyLane` استفاده می‌کنیم تا مفاهیم انتزاعی کوانتوم را به کدهای اجرایی در محیط کلاسیک تبدیل کنیم.

۱. کار با `quimb`: مدیریت انفجار ابعاد با شبکه‌های تنسوری

کتابخانه‌ی `quimb` سلطان مدیریت شبکه‌های تنسوری (Tensor Networks) است. زمانی که می‌خواهید یک سیستم با تعداد زیادی اجزا (مثلاً ۱۰۰ عامل هوشمند) را مدل‌سازی کنید، نباید از بردار حالت کامل استفاده کنید.

استراتژی پیاده‌سازی با Matrix Product States (MPS):

به جای یک ماتریس غول‌آسا، سیستم را به زنجیره‌ای از تنسورهای کوچک تبدیل می‌کنیم.

- مزیت:** حافظه به جای رشد نمایی (2^N)، به صورت خطی (N) رشد می‌کند.

نمونه کد مفهومی (English):

```
import quimb.tensor as qtn

# ایجاد یک زنجیره از ۱۰۰ سایت (کیوبیت مجازی)
n = 100
# محدود (برای کنترل حافظه) Bond Dimension تصادفی با MPS ساخت یک حالت
mps = qtn.MPS_rand_state(n, bond_dim=10)

# اعمال یک اپراتور محلی (مثلاً تغییر رفتار یک عامل)
# gate = ... (یک ماتریس ۴ در ۴)
# mps.gate(gate, (10, 11)) # ۱۱ و ۱۰ اعمال گیت روی عامل
```

نکته حرفه‌ای: در `quimb` نسخه‌ی ۲۰۲۶، قابلیت `lazy contraction` به شما اجازه می‌دهد عملیات را تعریف کنید اما تا زمانی که واقعاً به عدد نهایی نیاز ندارید، آن‌ها را اجرا نکنید. این کار باعث صرفه‌جویی عظیم در RAM می‌شود.

۲. شبیه‌سازی با `PennyLane`: استفاده از قدرت GPU

`PennyLane` به شما اجازه می‌دهد مدارهای کوانتومی را مانند لایه‌های یک شبکه عصبی آموزش دهید.

استفاده از پلاگین lightning.qubit :

این پلاگین با استفاده از ++C و OpenMP بهینه شده است. اگر از کارت گرافیک NVIDIA استفاده می‌کنید، lightning.gpu می‌تواند شبیه‌سازی را تا ۳۰ کیوبیت (حدود ۱ میلیارد حالت) به راحتی انجام دهد.

چالش لبه (Edge Case):

وقتی تعداد کیوبیت‌ها از ۲۵ بالاتر می‌رود، زمان انتقال داده بین CPU و GPU (Bottleneck) از زمان خود محاسبه بیشتر می‌شود.

- **راهکار:** از (Just-In-Time compilation) `qjit` استفاده کنید تا کل مدار شما به یک فایل باینری بهینه تبدیل شود و روی سخت‌افزار قفل شود.

۳. مدل‌سازی تصمیم‌گیری (Busemeyer & Bruza)

چگونه از ریاضیات کوانتوم برای مدل‌سازی «تصمیمات انسانی» استفاده کنیم؟ در مدل‌های کلاسیک، ما از احتمالات شرطی بیزی استفاده می‌کنیم، اما در مدل‌های کوانتومی از **دامنه احتمالات (Amplitudes)**.

پیاده‌سازی اثر سیاق (Context Effect):

در مدل Busemeyer، ترتیب سوالات بر پاسخ تأثیر می‌گذارد (A سپس B ≠ B سپس A).

- **کدنویسی:** به جای متغیرهای تصادفی، از **اپراتورهای پروجکشن (Projection Operators)** استفاده کنید.
- **فرمول کلیدی:** احتمال مشاهده حالت y بعد از حالت x برابر است با: $||P_y P_x \psi||^2 P(y|x) =$

۴. اشتباهات رایج در کار با این ابزارها

- **اشتباه آماتور:** استفاده از `default.qubit` در PennyLane برای کارهای سنگین. این شبیه‌ساز پایتونی است و به شدت کند است. همیشه از `lightning.qubit` استفاده کنید.
- **اشتباه حرفه‌ای:** نادیده گرفتن `Canonical Form` در شبکه‌های تنسوری. اگر تنسورهای خود را مرتب (Orthogonalize) نکنید، خطاهای عددی در ضرب‌های متوالی باعث می‌شود نتایج شما کاملاً غلط از آب دربیاید.

۵. تمرین عملی: شبیه‌سازی یک “بازار کوانتومی”

۱. با استفاده از `quimb` یک شبکه تنسوری ۲ بعدی بسازید که هر گره آن یک خریدار/فروشنده باشد. ۲. ارتباط بین آن‌ها را با درهم‌تنیدگی (Entanglement) مدل کنید (یعنی تصمیم یکی مستقیماً بر فضای احتمالات دیگری

تأثیر بگذارد). ۳. با استفاده از `quimb.tensor.fitting` سعی کنید پارامترهای بازار را به گونه‌ای تنظیم کنید که به یک تعادل پایدار برسد.

دستورات ترمینال برای نصب ابزارهای ۲۰۲۶:

```
pip install quimb pennylane pennylane-lightning[gpu]
```

سوال برای تفکر:

اگر در یک سیستم تصمیم‌گیرنده، «عدم قطعیت» را به جای نویز، به عنوان یک «ابریوزیسیون» مدل کنیم، آیا سیستم ما در مواجهه با شرایط پیش‌بینی نشده (Black Swan Events) مقاوم‌تر خواهد بود؟